# Custom Domain Architecture & Implementation Guide

Complete technical guide for connecting, verifying, and routing real custom domains to funnels & websites in Next.js & Firebase

---

**Target Framework:** Next.js (App Router) + Firebase Firestore

**Module:** Funnels & Public Sites Custom Domains

**Distribution Package:** `Custom_Domain_Feature_Files.zip`

## 1. Executive Summary & Core Objective

This documentation details the complete, production-tested architecture for allowing users to attach their own custom domain (e.g. `offers.clientbrand.com` or apex `clientbrand.com`) to any funnel or website created inside the platform.

The system achieves three critical objectives:

- **Transparent Edge Rewriting:** When visitors navigate to the custom domain, the URL in the browser remains `https://offers.clientbrand.com/slug` with full SSL encryption, while Next.js internally maps the request to the corresponding funnel renderer.
- **Automated Provisioning & Verification:** Node.js standard DNS queries verify that the user's CNAME or A records are active. The backend automatically registers the domain with Vercel API for instant SSL issuance and whitelists it in Firebase Auth.
- **Multi-Tenant Collision Prevention:** Domains are indexed in Firestore so no two accounts can claim the same domain, and platform root hostnames (e.g. `upklick.net`, `app.upklick.net`) are strictly guarded.

## 2. End-to-End System Architecture



## 3. Complete File Inventory

| File Path | Type / Layer | Main Responsibilities |
| --- | --- | --- |
| `src/proxy.js` | EDGE PROXY | Intercepts all incoming HTTP requests. Excludes platform domains and rewrites custom domain traffic to `/preview-site`. |
| `src/app/api/sites/verify-domain/route.js` | BACKEND API | Performs real-time DNS resolution (CNAME/A records), provisions SSL on Vercel, and authorizes domain in Firebase Auth. |
| `src/lib/sites/publicSite.js` | CORE LIBRARY | Data normalizer, Firestore operations ( `connectFunnelDomain` , `publishFunnelPublic` , `markDomainStatus` ). |
| `src/components/sites/DomainSettings.js` | REACT UI | Settings interface showing CNAME/A instructions, live DNS diagnostic tester, and disconnect confirmation modal. |
| `src/app/preview-site/page.js` | APP ROUTER | The target route of the rewrite. Receives the rewritten search params ( `host` , `path` ) and renders `LiveSiteView` . |
| `src/components/sites/LiveSiteView.js` | RUNTIME RESOLVER | Queries `site_domains` to resolve `funnelId` , matches the path to the page step, and renders the layout. |
| `firestore.rules` | SECURITY | Ensures public read access for visitors, while restricting create/update/delete strictly to the authenticated site owner. |
| `.env.example` | CONFIGURATION | Specification for DNS targets (CNAME host, A-record IP), Vercel tokens, and platform domain exclusions. |

## 4. Detailed Implementation by Layer

### A. Edge Proxy & Transparent Rewriting ( `src/proxy.js` )

In Next.js, the middleware/proxy intercepts requests at the edge. The function `isPlatformHost(host)` distinguishes platform routes from custom customer domains:

```
import { NextResponse } from 'next/server';

function hostnameOf(hostHeader) {
  return String(hostHeader || '').split(':')[0].trim().toLowerCase();
}

function isPlatformHost(host) {
  if (!host) return true;
  const clean = hostnameOf(host);
  if (['localhost', '127.0.0.1'].includes(clean)) return true;
  if (clean.endsWith('.vercel.app')) return true;
  if (clean === 'upklick.net' || clean.endsWith('.upklick.net')) return true;
  return false;
}

export function proxy(request) {
  const host = hostnameOf(request.headers.get('host'));
  if (isPlatformHost(host)) return NextResponse.next();

  const url = request.nextUrl.clone();
  if (url.pathname.startsWith('/preview-site') || url.pathname.startsWith('/api/') || url.pathname.st
    return NextResponse.next();
  }

  // Rewrite customer domain to preview-site handler
  url.pathname = '/preview-site';
  url.searchParams.set('host', host);
  url.searchParams.set('path', request.nextUrl.pathname || '/');

  const requestHeaders = new Headers(request.headers);
  requestHeaders.set('x-custom-domain', host);
  requestHeaders.set('x-custom-path', request.nextUrl.pathname || '/');

  return NextResponse.rewrite(url, {
    request: { headers: requestHeaders }
  });
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico|robots.txt|.*\\..*).*)']
};
```

### B. Firestore Collections & Data Schemas

Two primary Firestore collections manage the routing state:

- **Collection `site_domains`:** Document ID is the normalized domain (e.g. `offers.clientbrand.com`).

```
{
  "host": "offers.clientbrand.com",
  "funnelId": "fn_abc123456",
  "ownerUid": "user_xyz789",
  "status": "connected", // or "pending"
  "checkedAt": "2026-10-22T15:00:00Z",
  "updatedAt": "2026-10-22T14:30:00Z"
}
```

- **Collection `published_sites`:** Document ID is the `funnelId`. Stores published steps, canvas elements, and settings.

### C. DNS Verification & Vercel SSL Automation (`route.js`)

When the user clicks "Check DNS", `POST /api/sites/verify-domain` is called with `{ host }`:

- **CNAME Verification:** Uses `dns.promises.resolveCname(host)` to verify it resolves to `cname.vercel-dns.com`.
- **A-Record Verification:** Uses `dns.promises.resolve4(host)` to verify it points to `76.76.21.21`.
- **Vercel Domain Provisioning:** Sends a `POST` request to `https://api.vercel.com/v10/projects/{projectId}/domains` using `VERCEL_TOKEN`. Vercel provisions the domain and generates an SSL certificate automatically.
- **Firebase Auth Whitelisting:** Uses Google Cloud Identity Toolkit API to append the host to `authorizedDomains` so OAuth authentication works on the custom domain.

### D. UI Component ( `DomainSettings.js` )

A high-converting, user-friendly settings component that renders inside the Funnel Settings modal:

- Displays the default free production URL ( `https://upklick.net/s/{funnelId}` ).
- Domain input field with instant normalization.
- **Interactive DNS Guidance Tab:**
  - **CNAME Tab:** For subdomains (e.g. `offers` ). Points to `cname.vercel-dns.com` .
  - **A-Record Tab:** For apex domains (e.g. `@` ). Points to `76.76.21.21` .
- One-click copy buttons for record values.
- Diagnostic status badge with real-time feedback (CNAME detected, IPs detected).
- Modal confirmation before disconnecting a domain, preventing accidental deletion.

### E. Runtime Resolver ( `LiveSiteView.js` )

When a visitor loads any page on the custom domain:

```
// 1. Resolve host from incoming request
const incomingHost = normalizeHost(host || window.location.hostname);

// 2. Query Firestore site_domains to obtain funnelId
let domainSnap = await getDoc(doc(db, 'site_domains', incomingHost));
if (domainSnap.exists()) {
  resolvedFunnelId = domainSnap.data().funnelId;
}

// 3. Query published site data
const siteSnap = await getDoc(doc(db, 'published_sites', resolvedFunnelId));
const funnelData = siteSnap.data();

// 4. Match URL path with step path (e.g. "/" or "/checkout")
const activeStep = pickPublishedStep(funnelData, { path: effectivePath });

// 5. Render elements canvas
return <ElementRenderer elements={activeStep.canvas} />;
```

## 5. Step-by-Step Implementation Guide for Your Colleague

1. **Extract the Package:** Unzip `Custom_Domain_Feature_Files.zip` into the target Next.js repository root.
2. **Configure Environment Variables:** Copy values from `.env.example` into your `.env.local` and production deployment environment (Vercel / AWS / Docker).
3. **Update Firestore Security Rules:** Add the rules from `firestore.rules` to allow public reading of `site_domains` and `published_sites` while restricting writes to authenticated owners.
4. **Deploy to Hosting with Custom Domain Support (e.g. Vercel):** Ensure your hosting provider supports wildcard or dynamic custom domain additions via API (Vercel provides this natively with zero server management).

5. **Mount the UI:** In your Funnel / Website Settings page, import `DomainSettings`:

```
<DomainSettings
  funnel={funnel}
  ownerUid={user.uid}
  onSaveFunnel={(updates) => updateFunnelState(updates)}
  showToast={(msg) => triggerToast(msg)}
/>
```

**Summary & Recommendation**

All source files in `Custom_Domain_Feature_Files.zip` are decoupled, clean, and pre-configured. They follow standard Next.js App Router patterns and can be dropped directly into any compatible application.